From OpenAPI to the Wire: Specification-Grounded Conformance Testing of Generated API Clients

Anonymous Author(s)

Affiliation withheld for review

Abstract. Generated clients are widely treated as executable projections of OpenAPI contracts, yet the request they place on the wire can differ from the contract's parameter-serialization rules. We present WireConform, a specification-grounded method that generates clients, calls their typed SDK operations, captures HTTP requests locally, and compares paths, query multisets, and headers with an independently checked oracle. Its 117-case OpenAPI 3.1.1 corpus covers seven serialization styles, three parameter locations, three value profiles, and 27 implicit/explicit-default pairs. We evaluate OpenAPI Generator TypeScript Fetch 7.26.0, Orval Fetch 8.42.0, and Hey API Fetch 0.99.0. All 117 OpenAPI Generator cases, 93 Orval cases, and 111 Hey API cases produced eligible observations. Their respective pass counts were 54/117, 32/93, and 58/111. The remaining eligible observations were reproducible wire mismatches; Orval omitted typed header parameters in 24 generated signatures, while Hey API emitted uncompilable source for six valid delimited-object cases. All eligible default pairs were equivalent, showing that default expansion alone did not reveal the prevalent faults. Every one of the 177 wire mismatches was regenerated from its individual specification and reproduced exactly. The results reveal systematic blind spots for label and matrix paths, object-valued parameters, and delimiter encoding, and demonstrate why source generation and compilation are insufficient conformance evidence.

Keywords: OpenAPI · code generation · conformance testing · parameter serialization · software quality

1 Introduction

OpenAPI descriptions are routinely used to generate client SDKs. A generated method may type-check and execute while still constructing a request that violates the contract. Under query style `form` with `explode=true`, for example, an object `{a:red,b:blue}` is represented as `a=red&b=blue`; nesting it under the parameter name is a different request [1]. Such discrepancies can remain hidden when servers accept several encodings.

Parameter serialization is a useful conformance target because its inputs are small, the expected wire representation is defined, and the emitted request is

directly observable. Existing OpenAPI testing work primarily derives tests for running services [2,3,4,5]. We instead test the client as the system under test: the outcome is the exact path, query, or header emitted after invoking a generated SDK operation.

We contribute (1) a 117-case, explicitly bounded OpenAPI 3.1.1 benchmark and dual-implementation oracle; (2) an execution protocol that distinguishes serialization failures from unsupported signatures and generated-source failures; (3) a three-generator evaluation using genuine generated SDK calls and captured requests; and (4) individual-specification reproduction of every mismatch. The study also tests a metamorphic relation between omitted serialization defaults and the same defaults written explicitly.

2 Semantics and Research Questions

We model one parameter as $p = (\ell, s, e, t, n)$, where ℓ is its location, s its style, e its `explode` value, t its value type, and n its name. Values are nonempty RFC 3986-unreserved ASCII tokens, arrays of such tokens, or flat objects with token-valued fields. Each operation has exactly one tested parameter. This restriction isolates serialization from nulls, nesting, name collisions, authentication, and server normalization.

The corpus includes query styles `form`, `spaceDelimited`, `pipeDelimited`, and `deepObject`; path styles `simple`, `label`, and `matrix`; and header style `simple`. It excludes combinations for which the specification does not define behavior, nested `deepObject`, cookies, and `allowReserved=true`. Query parameters default to `form`, path and header parameters to `simple`; `explode` defaults to true only for `form` [1].

Let $\sigma(p, v)$ be the expected wire representation. For generator configuration G, invoking the generated operation yields $W(G, p, v) = (P, Q, H)$: request path P, a multiplicity-preserving but order-insensitive multiset Q of decoded query pairs, and relevant case-insensitive headers H. Raw targets are retained for encoding review because decoding can hide whether structural delimiters were percent-encoded. A case passes only when all applicable components and required encoding agree.

For a declaration p omitting defined serialization defaults, let $T(p)$ add those defaults without changing anything else. Their effective semantics imply

$$\sigma(p, v) = \sigma(T(p), v). \tag{MR-D}$$

We ask: **RQ1** What conformance and support are observed by location, style, and value type? **RQ2** Do implicit and explicit defaults yield equivalent requests? **RQ3** Which discrepancies reproduce from individual specifications, and what failure families characterize them?

3 Method

3.1 Corpus and oracle validation

A deterministic builder creates one self-contained OpenAPI 3.1.1 document per case. Three value profiles exercise multi-element words, numeric values, and singleton values. The resulting 117 cases contain 39 query, 54 path, and 24 header cases, including 27 MR-D pairs. Expected representations follow the Parameter Object and style examples in OpenAPI 3.1.1 [1]. This patch version matters because it corrects non-exploded label examples that conflicted with RFC 6570 in version 3.1.0.

We used four checks before evaluating generators. First, 33 hand-transcribed standards examples pass fixed golden tests. Second, an independently engineered JavaScript implementation agrees with the Python oracle for all 117 cases. Third, an independent structural audit finds no invalid style/type combination, duplicate operation signature, or malformed default pair. Fourth, seven injected-fault operators test the scorer without contributing any generator observations: the corpus detects 21/25 applicable instances; four singleton arrays inherently conceal delimiter faults. A 14-case pilot selected before external observations covers all seven modeled fault classes. In total, 60 automated tests pass. Redocly validates the merged document without errors (documentation warnings are retained in the artifact).

3.2 Generation, invocation, and capture

To make generation practical, we also construct a single 117-operation document and verify operation-level equivalence with every individual document. We invoke pinned tools: OpenAPI Generator CLI 2.41.0 with generator JAR 7.26.0 and target `typescript-fetch`; Orval 8.42.0 with its Fetch client; and `@hey-api/openapi-ts` 0.99.0 with `@hey-api/client-fetch` 0.13.1. Node.js 24.17.0 and TypeScript 6.0.3 are used on Windows 11.

The adapter imports each generated SDK, supplies the typed parameter value to the generated operation, and points the client's ordinary transport at a local capture server. It neither constructs requests nor serializes parameters. Tool-specific source audits record the called symbol, argument path, generated-source hash, and adapter hash. For Orval, the generated Fetch function exposes no typed header parameter; injecting `RequestInit.headers` would bypass the feature under test, so those 24 cases are classified as unsupported generated signatures. Hey API's generated source fails TypeScript compilation for the six object-valued `spaceDelimited`/`pipeDelimited` cases, which are valid with `explode=false`; those are compilation failures, not wire mismatches.

The capture server assigns each invocation a unique path prefix and records its raw request target and headers. A wire observation is eligible only when generation succeeded, the audited generated method was invoked, its process succeeded, and exactly one request was captured. Mismatches are not attributed from the merged build alone: each is regenerated from the original one-operation

Table 1. Finite-corpus results. CI is the descriptive 95% Wilson interval for pass/eligible.

Tool	Built	Elig.	Pass	Fail	Rate (95% CI)
OpenAPI Generator	117	117	54	63	.462 [.374,.552]
Orval	117	93	32	61	.344 [.255,.445]
Hey API	117	111	58	53	.523 [.430,.613]

document and rerun through a separate single-case adapter. We require exact agreement between bundled and individual raw targets or tested headers.

3.3 Analysis

We report support coverage $N_{\text{eligible}}/117$ separately from conditional conformance $N_{\text{pass}}/N_{\text{eligible}}$. Wilson intervals are descriptive summaries of this finite corpus, not population inferences. Generation, compilation, unsupported-signature, capture, and serialization outcomes remain distinct. The benchmark was fixed before real generator observations; synthetic fixtures are excluded from all generator counts.

4 Results

4.1 RQ1: support and conformance

Table 1 reports complete counts. OpenAPI Generator generated and executed every operation but passed 54/117. Orval generated all operations, yet its typed API omitted all 24 header parameters; among 93 eligible requests, 32 passed. Hey API had six generated-source compilation failures and passed 58/111 eligible requests. Thus generation success alone substantially overstates usable and conforming coverage.

Failures were systematic rather than isolated (Table 2). All three tools failed all 15 label-path and all 15 matrix-path cases. OpenAPI Generator failed every flat-object case except `deepObject`; its 63 failures comprise label (15), matrix (15), and six object families totaling 33. Orval also failed simple-path and form-query objects, all three `deepObject` cases because brackets were left unencoded, and multi-element delimited arrays. Hey API correctly handled all nine form-query objects, but failed multi-element header arrays, all header objects, three simple-path objects, all `deepObject`, and multi-element pipe arrays. Singleton values explain why some otherwise faulty delimiter families contain passes.

Representative captures show distinct causes. For exploded form object case `c007`, the expected query is `a=red&b=blue`; OpenAPI Generator emits deep-object-shaped keys, Orval stringifies the object, and Hey API passes. For exploded label object `c021`, the expected path segment is `.a=red.b=blue`; OpenAPI Generator and Orval contain `[object Object]`, while Hey API emits a

Table 2. Confirmed wire mismatches by feature family; each entry is fail/eligible.

Family	OpenAPI Gen.	Orval	Hey API
Path label (all values)	15/15	15/15	15/15
Path matrix (all values)	15/15	15/15	15/15
Path simple, object	9/9	9/9	3/9
Query form, object	9/9	9/9	0/9
Query deepObject, object	0/3	3/3	3/3
Query pipe, array	0/3	2/3	2/3
Query space, array	0/3	2/3	0/3
Header simple, array/object	9/18	–	15/18
Delimited query, object	6/6	6/6	–

comma-flattened sequence. For header object `c034`, OpenAPI Generator stringifies the object, Hey API sends JSON-like text, and Orval exposes no typed argument. For pipe array `c037`, the structural pipe must be encoded when `allowReserved=false` [1]; OpenAPI Generator does so, Orval uses a comma, and Hey API leaves a literal pipe.

4.2 RQ2: default equivalence

Every pair whose two members were eligible satisfied MR-D: 27/27 for OpenAPI Generator, 18/18 for Orval, and 27/27 for Hey API. Orval's other nine pairs were header cases and therefore ineligible. This is a useful negative result: the tools treated omitted and explicit defaults consistently, but often serialized both variants incorrectly. Absolute specification oracles are therefore necessary; metamorphic consistency cannot establish correctness by itself. The Orval release tested already contains the fix for its earlier default-explode issue [7], consistent with this result.

4.3 RQ3: independent reproduction and classification

All 177 confirmed mismatches—63 OpenAPI Generator, 61 Orval, and 53 Hey API—were reproduced from clean, individual specifications. In every case the raw request target or tested header exactly matched its bundled counterpart. The six Hey API compile failures also recur in isolated generation. This removes merged-specification interaction as an explanation for the reported outcomes.

The observations cluster into four mechanisms: unsupported style syntax (especially label and matrix); generic JavaScript coercion of objects to `[object Object]`; style-insensitive flattening or JSON-like serialization; and incorrect delimiter percent-encoding. Related issue reports predate parts of these classes: OpenAPI Generator issue 22252 documents exploded form-object nesting [8]; Hey API issue 393 requested query-style support [6]; and Orval issue 3231 noted that its Fetch path ignored styles beyond a default-explode fix [7]. We therefore claim current-version, cross-feature reproduction and measurement, not novelty for every underlying defect.

5 Threats to Validity

Construct validity. The oracle implements a deliberately restricted fragment. Its agreement with fixed standards examples and a separate implementation reduces, but does not eliminate, shared interpretation error. Raw-target review avoids accepting unencoded structural delimiters after percent-decoding. Server acceptance is intentionally irrelevant: the construct is contract conformance.

Internal validity. Adapters could accidentally serialize inputs. Source audits show that they call generated operations and only configure transport/base URL. All mismatches reproduce through a second per-case path. We classify missing signatures and compiler failures separately rather than forcing requests through escape hatches. Tool versions, hashes, logs, captures, and case-level results are retained.

External validity. Results apply to the named versions, Fetch targets, configuration, runtime, and 117-case domain. They do not characterize other languages, templates, browser behavior, nested or reserved values, cookies, multiple interacting parameters, or deployment prevalence. A finite corpus pass fraction is not an ecosystem-wide reliability estimate. Conversely, the use of simple values may hide further defects; singleton arrays already mask four injected faults.

6 Related Work

QuickREST derives property-based tests from OpenAPI and checks running services [2]. RESTest and RestTestGen generate nominal and error scenarios for REST APIs [3,4], while Corradini et al. compare black-box REST testing tools empirically [5]. Those approaches establish API descriptions as test-generation substrates. WireConform changes the subject and oracle: it tests generated client request construction and judges serialization before application behavior. Public generator issues provide important prior defect evidence [6,7,8]; our protocol adds a fixed cross-generator corpus, exact wire capture, explicit exclusions, and clean-case reproduction.

7 Conclusion

Across three current TypeScript Fetch generators, only 144 of 321 eligible requests matched the restricted OpenAPI 3.1.1 oracle; 177 mismatched, 24 more lacked typed header parameters, and six failed compilation. All eligible implicit/explicit-default pairs were internally consistent, so metamorphic testing alone would have missed the dominant defects. Exact per-case reproduction and separation of support from correctness turn these observations into auditable findings rather than inferred source-code concerns. Generated clients should be tested at the wire boundary, with unsupported and failed cases reported alongside pass rates.

Data Availability Statement

The submission artifact contains the complete deterministic corpus, oracle and independent reference, pinned dependency lockfile, generated SDKs, adapter audits, raw captures, compilation and execution logs, case-level CSV/JSON results, individual mismatch reproductions, and commands for validation and rerun. The artifact accompanying this manuscript has SHA-256 manifest coverage and contains no synthetic observation in generator statistics. A public archival URL cannot be supplied during this local preparation stage and must be inserted if required by the submission system.

References

1. OpenAPI Initiative: OpenAPI Specification v3.1.1 (2024). https://spec.openapis.org/oas/v3.1.1.html
2. Karlsson, S., Čaušević, A., Sundmark, D.: QuickREST: Property-based test generation of OpenAPI-described RESTful APIs. In: ICST (2020). https://doi.org/10.1109/ICST46399.2020.00051
3. Martin-Lopez, A., Segura, S., Ruiz-Cortés, A.: RESTest: Automated black-box testing of RESTful web APIs. In: ISSTA (2021). https://doi.org/10.1145/3460319.3469082
4. Corradini, D., et al.: Automated black-box testing of nominal and error scenarios in RESTful APIs. Softw. Test. Verif. Reliab. (2022). https://doi.org/10.1002/stvr.1808
5. Corradini, D., Zampieri, A., Pasqua, M., Ceccato, M.: Empirical comparison of black-box test case generation tools for RESTful APIs. In: SCAM (2021). https://doi.org/10.1109/SCAM52516.2021.00032
6. Hey API: Query parameter serialization styles, issue 393 (2024). https://github.com/hey-api/hey-api/issues/393
7. Orval: Fetch client does not respect OpenAPI default explode, issue 3231 (2026). https://github.com/orval-labs/orval/issues/3231
8. OpenAPI Generator: Incorrect arbitrary query parameters via form/explode, issue 22252 (2025). https://github.com/OpenAPITools/openapi-generator/issues/22252